Do neural and zero-shot models beat persistence in stock price forecasting? A cross-market study

Chu-Nuo Tan[1] Himat Shah[2*]

[1]Guangzhou Maritime University, Guangzhou, China

[2]Robotics and Artificial Intelligence, Centria University of Applied Sciences, Vierimaantie 7, 84100 Ylivieska, Finland

Abstract

This study compares selected neural and zero-shot time-series models with latest-close persistence for stock-price forecasting across markets. This simple baseline uses the latest observed closing price to predict future price levels. Daily data from 2015 to April 2026 cover stocks in the United States (US) and Hong Kong and a global exchange-traded fund (ETF) basket. The five lists contain 165 asset entries and 133 unique tickers. The experiments cover two price modes and three forecast horizons, with each model and its baseline sharing the same test origins and target dates. The supervised implementations include long short-term memory (LSTM), gated recurrent unit (GRU), a linear temporal proxy, and compact PatchTST-style, iTransformer-style, and TimeMixer-style models. Only 39 of 35,640 supervised configuration–seed rows achieve lower root mean squared error (RMSE) than persistence, and every implementation has a median RMSE ratio above one. Chronos-T5-small passes in 119 of 990 sampled-origin rows, with a median ratio of 1.036. Its different protocol prevents a direct ranking against the supervised models. These descriptive results support a bounded conclusion: most selected configurations do not improve on persistence for absolute price-level prediction under the tested protocols. The study provides a baseline-first evaluation with dated forecasts and explicit limits on cross-model comparisons.

Keywords: stock forecasting; persistence baseline; time-series foundation models; Chronos; PatchTST; iTransformer; TimeMixer; negative results

1 Introduction

Stock-price forecasting asks a simple question: how much of a future price can past observations explain? For price-level prediction, the latest observed close already provides a forecast without model training. This rule, known as latest-close persistence, is a standard naive forecasting method [7]. A learned model must improve on this comparator before its lower error against another learned model establishes an advantage over persistence.

The target and evaluation design determine what such an advantage means. Predicting a future price level differs from predicting its direction, an excess return, or a profitable trading decision. A model and its baseline also need the same forecast origins and target dates for a meaningful error comparison. Different feature requirements can otherwise change the usable observations and the resulting test period.

*Corresponding author.

This study asks whether selected neural and zero-shot time-series models beat persistence in cross-market price-level prediction. The experiment grid covers 165 asset entries, representing 133 unique tickers across five configured lists. It includes two price modes and three forecast horizons, with several compact supervised implementations and Chronos-T5-small. Only 39 of 35,640 supervised configuration–seed rows achieve lower RMSE than matched persistence, and every supervised implementation has a median RMSE ratio above one. Chronos has local wins, but its sampled-origin median also remains above one; its different protocol prevents a direct ranking against the supervised models.

The paper makes three contributions:

- A baseline-first evaluation that pairs each model forecast with persistence on the same origins and target dates.
- A cross-market comparison of selected compact implementations and one small zero-shot model, with separate reporting for different evaluation protocols.
- Corrected evaluation records with dated forecasts, explicit experiment keys, and descriptive summaries that make the scope of the negative result clear.

The following sections review related work, describe the corrected protocol, and report the results. The discussion and limitations explain which conclusions the experiment supports. The reproducibility sections distinguish the corrected local evidence from the earlier public archive.

2 Related work

Financial forecasting studies differ in their targets, benchmarks, and use of training data. A gain in direction classification does not establish a gain in price-level regression, and a zero-shot result does not describe every foundation model. Table 1 summarizes these distinctions in the work most relevant to this study. The review then connects financial targets, forecasting architectures, and model adaptation to the present audit.

Table 1: Targets and evaluation scope in related work.

Study	Target or scope	Main approach	Relation to this audit
Bao et al. [1]	Future closing price	Hybrid deep learning	A price-level task; the present audit centers the persistence comparison.
Patel et al. [2]	Price direction	Indicators and classifiers	A different target from absolute price regression.
Campbell and Thompson; Welch and Goyal [5, 6]	Excess returns and equity premium	Predictive regressions and historical-average benchmarks	Relevant benchmark lessons, with different targets and findings.
Zeng et al.; Nie et al.; Liu et al.; Wang et al. [11, 12, 13, 14]	General time-series forecasting	Linear, patch, variate-token, and multiscale designs	Architectural context; compact proxies do not reproduce the full published systems.
Ansari et al.; Goswami et al. [15, 18]	Forecasting and general time-series analysis	Pre-training with different adaptation settings	Foundation models have different scopes; this audit tests only Chronos-T5-small.

2.1 Financial targets and benchmarks

Bao et al. combine wavelet processing, stacked autoencoders, and LSTM for future closing-price prediction [1]. Patel et al. instead study price movement using technical indicators and four classifiers [2]. Their classification target differs from the absolute price-level target here. Sezer et al. review deep learning for financial time-series forecasting, while Ozbayoglu et al. survey a wider range of financial applications [3, 4]. These studies provide context for model and feature choices, but their different tasks do not form a common persistence benchmark.

Simple benchmarks also play a central role in return prediction. Welch and Goyal document out-of-sample difficulties in equity-premium prediction relative to a historical-average benchmark [6]. Campbell and Thompson show that weak restrictions can improve predictive regressions and allow many of them to beat that benchmark [5]. Their findings demonstrate why the evaluation design matters, rather than a universal failure of learned forecasting. Both studies concern returns, so neither directly establishes a result about future absolute closing prices.

General forecasting research provides a further reason to keep simple comparators visible. Hyndman and Athanasopoulos define the naive forecast as the latest observation and use it as a benchmark [7]. Makridakis et al. compare machine-learning and statistical methods on monthly competition series and emphasize broad, explicit benchmark evaluation [8]. Their experiment concerns a different collection of series and methods. We adopt the evaluation principle, while deriving the stock-price conclusions from the present experiment.

2.2 Forecasting architectures

Deep forecasting surveys describe recurrent, convolutional, attention-based, and other model families [9, 10]. Recent architectures change how models represent observations rather than changing the need for a suitable benchmark. DLinear combines moving-average decomposition with separate linear projections of the resulting components [11]. PatchTST uses patches and channel-independent modeling, while iTransformer represents individual variates as tokens [12, 13]. TimeMixer combines decomposition with mixing across temporal scales [14].

The present audit uses these papers to motivate several architectural motifs. Its supervised implementations include recurrent models, a linear temporal proxy, and compact patch-based, variate-token, and multiscale proxies. The linear proxy omits the decomposition in DLinear, and the other proxies do not reproduce all components or settings of the corresponding published systems. Their results therefore describe these implementations under the present training protocol. They do not rank the official architectures or establish their best achievable performance.

2.3 Foundation models and adaptation

Chronos scales and quantizes time-series values for pre-trained probabilistic forecasting [15]. Lag-Llama, TimesFM, and Moirai also report zero-shot forecasting results, using different model designs and training collections [16, 17, 19]. MOMENT covers a broader set of time-series analysis tasks and evaluates adaptation with limited supervision and task-specific fine-tuning [18]. These models do not share one interchangeable training or adaptation protocol. Success on their benchmark tasks alone does not establish an advantage over persistence for the stock-price target here.

This study evaluates only Chronos-T5-small without asset-specific fine-tuning. Its contexts, input variables, and sampled test origins differ from those of the supervised models. We therefore report its results separately and use a small full-origin subset as a limited sensitivity check. The central question is whether each selected implementation improves on its own matched persistence

baseline. The experiment does not test the other foundation models or provide a controlled ranking across model families.

3 Audit protocol

The audit compares future closing-price forecasts with latest-close persistence. The protocol specifies data preparation, time splits, model fitting, and evaluation. Each corrected model–baseline comparison uses identical test origins and target dates. The subsections below describe these steps and distinguish the supervised and zero-shot protocols.

Figure 1 outlines the two evaluation paths and their shared persistence comparison.

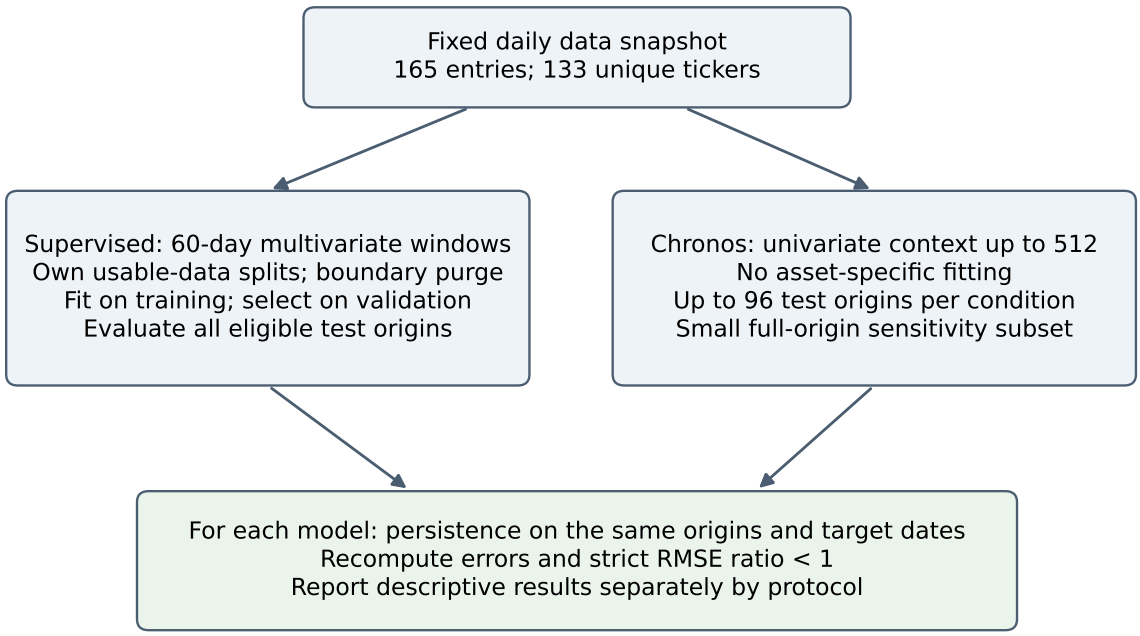


Figure 1: Workflow of the corrected audit.

3.1 Data collection

The audit uses a fixed snapshot of daily Yahoo Finance data spanning 02-01-2015 to 30-04-2026. Table 2 summarizes the five configured asset lists. The download summary contains 165 asset entries and 133 unique ticker symbols because several stocks appear in more than one list. One configured Hong Kong ticker has no successful download and does not enter the analysis. The overall date range does not imply that every asset has a complete record throughout this period.

The inputs contain open, high, low, close, and volume (OHLCV). Raw-close experiments use the downloaded fields directly, while adjusted-close experiments multiply OHLC prices by the downloaded adjusted-close to close ratio and retain the volume field. The target is the closing price 1, 5, or 10 trading days after the forecast origin. Each origin represents the point after observing that day's close, and each horizon uses a separate direct forecast. These experiments evaluate price levels, not returns, directions, volatility, or trading profit.

Table 2: Data coverage.

Configured list	Asset entries	Unique tickers within list
Dow 30	30	30
ETF basket	12	12
Hang Seng large-cap	43	43
Nasdaq large-tech	30	30
S&P 100 subset	50	50

The five lists contain 165 entries but 133 unique ticker symbols across lists. The overall download span is 02-01-2015 to 30-04-2026; individual records can start later.

3.2 Time splits and persistence

For each asset, price mode, horizon, and feature set, we first remove rows with missing required features or targets. We divide the remaining observations chronologically into 72% training, 8% validation, and 20% testing. We then remove training and validation examples whose target dates reach or pass the first origin of the next segment. These percentages describe the split rule before this boundary removal. Indicator warmup and differences in available records mean that feature sets and assets can have different evaluation dates.

We fit feature min–max scalers using feature rows before the cut between training and validation. Training-target normalization uses only the retained training labels. Validation loss selects the checkpoint, and test errors do not guide checkpoint selection. At each test origin, the fitted model uses its available input window without another training update. The exact boundary file records the dates and retained sample counts for all 1,980 asset–price–horizon–feature conditions.

Latest-close persistence predicts the future closing price with the close at the forecast origin. This is the naive forecasting rule applied to the price-level target [7]. We compute each model's baseline errors on exactly the same test origins and target dates as its forecasts. This alignment ensures that their reported errors describe the same test cases. It does not make different feature sets or forecasting protocols a common-date model comparison.

3.3 Supervised implementations

The supervised experiments cover LSTM, GRU, a linear temporal proxy, and compact PatchTST-style, iTransformer-style, and TimeMixer-style implementations. Table 3 describes their main components. The linear proxy has no trend and remainder decomposition and therefore does not implement the published DLinear architecture. The three modern style implementations also differ from the official full systems. Our results concern these selected implementations rather than rankings of the published architectures.

Each model uses a 60-trading-day input window. The two feature sets are OHLCV and OHLCV with relative strength index (RSI), moving average convergence divergence (MACD), and Bollinger-band indicators. Recurrent and transformer representations have dimension 64, while the TimeMixer-style hidden layers have widths 128 and 64. The nonlinear implementations use dropout 0.1, and the transformer and mixing components use Gaussian error linear unit activations. The full grid contains 5,940 configuration–seed rows per implementation, or 35,640 supervised rows in total.

Training uses AdamW with learning rate 0.001, zero weight decay, batch size 128, and gradient clipping at norm 1. We allow at most 80 epochs and stop after eight epochs without a validation-loss improvement greater than 10^{-5}. We retain the best validation checkpoint and use seeds 42, 123, and 777. Training batches shuffle within the training segment; validation and test observations retain chronological order. Each fitted model then predicts all eligible test origins without refitting.

Table 3: Components of the supervised implementations.

Implementation	Main components
LSTM and GRU	One recurrent layer, dropout, and a linear output head.
Linear temporal proxy	A linear projection over time followed by a linear feature head; no decomposition.
PatchTST-style	Patches of length 10 and stride 5; two encoder layers and four attention heads.
iTransformer-style	Variables as tokens; two encoder layers and four attention heads.
TimeMixer-style	Concatenated representations at scales 1, 2, and 4; two hidden layers with widths 128 and 64.

3.4 Chronos evaluation

The zero-shot experiments use Chronos-T5-small without fitting it to the study assets. Each forecast uses a univariate close-price context of up to 512 observations and 20 generated sample paths. The point forecast uses the lower middle value among the 20 samples at the requested horizon. The main protocol selects up to 96 evenly spaced indices from each chronological test segment, producing 990 asset–price–horizon rows. The implementation fixes the model revision and derives a reproducible generation seed from base seed 42 and the experiment key.

A supplementary check uses every eligible test origin for the first two assets in each configured list. It covers both price modes and all three horizons, giving 60 rows from 10 asset entries but nine unique tickers. This small, non-random subset provides a limited sensitivity check. The sampled and full-origin runs generate forecasts separately, so their differences do not isolate the effect of origin selection alone. Chronos also differs from the supervised protocol in context length, input variables, and test-origin coverage, so we report the protocols separately.

3.5 Errors and descriptive summaries

For n target prices y_i and forecasts $\hat{y}_i$, we record RMSE, mean absolute error (MAE), and mean absolute percentage error (MAPE):

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{1}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \tag{2}$$

$$\mathrm{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{\max(|y_i|, 10^{-8})}. \tag{3}$$

The RMSE gate requires the model-to-persistence RMSE ratio to be strictly below one. Equality does not pass, and a separate all-three count requires lower RMSE, MAE, and MAPE. Model summaries report counts, rates, medians, and descriptive distributions of the fixed experiment grid. Repeated tickers, price modes, horizons, feature sets, and seeds create dependent rows, so these rows are not independent trials. We therefore do not report row-independent confidence intervals or significance tests.

4 Results

The corrected results compare each model with persistence on the same forecast origins and target dates. Table 4 summarizes the RMSE gate and the separate all-three criterion. Table 5 reports horizon-specific medians. We discuss supervised and zero-shot results separately because their inputs and test-origin protocols differ.

Table 4: Results of the persistence gate.

Implementation	Rows	RMSE wins	Rate (%)	All-three wins	Median ratio
LSTM	5,940	6	0.10	2	2.26
GRU	5,940	21	0.35	13	1.92
Linear temporal proxy	5,940	0	0.00	0	2.09
PatchTST-style	5,940	2	0.03	1	3.35
iTransformer-style	5,940	3	0.05	1	3.53
TimeMixer-style	5,940	7	0.12	3	2.25
Chronos sampled	990	119	12.02	67	1.04
Chronos full subset	60	1	1.67	1	1.03

RMSE wins require a ratio strictly below one; all-three wins require lower RMSE, MAE, and MAPE. Counts use full precision, while displayed ratios use two decimals. Chronos protocols differ from the supervised protocol and from each other. Rows are dependent configurations, not independent trials.

Table 5: Median RMSE ratios by horizon.

Implementation	1 day	5 days	10 days
LSTM	2.54	2.18	2.10
GRU	2.15	1.83	1.79
Linear temporal proxy	3.30	1.90	1.69
PatchTST-style	5.11	2.97	2.50
iTransformer-style	6.40	3.13	2.54
TimeMixer-style	3.28	2.07	1.88
Chronos sampled	1.04	1.03	1.03

Each ratio compares a model with persistence on its own test origins. Different feature and model protocols can retain different dates.

4.1 Supervised results

Only 39 of 35,640 supervised configuration–seed rows pass the RMSE gate, a rate of 0.11%. LSTM contributes six wins, GRU 21, the linear temporal proxy none, PatchTST-style two, iTransformer-style three, and TimeMixer-style seven. Twenty rows improve all three error measures. Every supervised implementation has a median RMSE ratio above one, ranging from 1.92 for GRU to 3.53 for iTransformer-style. Under this protocol, persistence therefore has lower error than most of the selected learned configurations.

The horizon summaries also show median ratios above one for every supervised implementation at 1, 5, and 10 days. None of the supervised rows passes at one day, while nine pass at five days and 30 at ten days. The 39 winning rows involve nine unique tickers and 29 distinct list–asset–price–horizon–feature–implementation conditions before seeds. Only one such condition passes with all three seeds: GRU on adjusted TLT prices, a ten-day horizon, and OHLCV inputs. These counts

describe local results in the fixed grid rather than independent evidence of a general forecasting advantage.

4.2 Chronos results

Chronos-T5-small passes in 119 of 990 sampled-origin rows, or 12.02%, and improves all three errors in 67 rows. Its median RMSE ratio is 1.036, so the median row remains slightly worse than persistence. Individual wins occur in several asset lists. The nominal win rate does not rank Chronos against the supervised models, because the contexts, variables, and forecast origins differ. Figure 2 shows the descriptive distributions on separate scales.

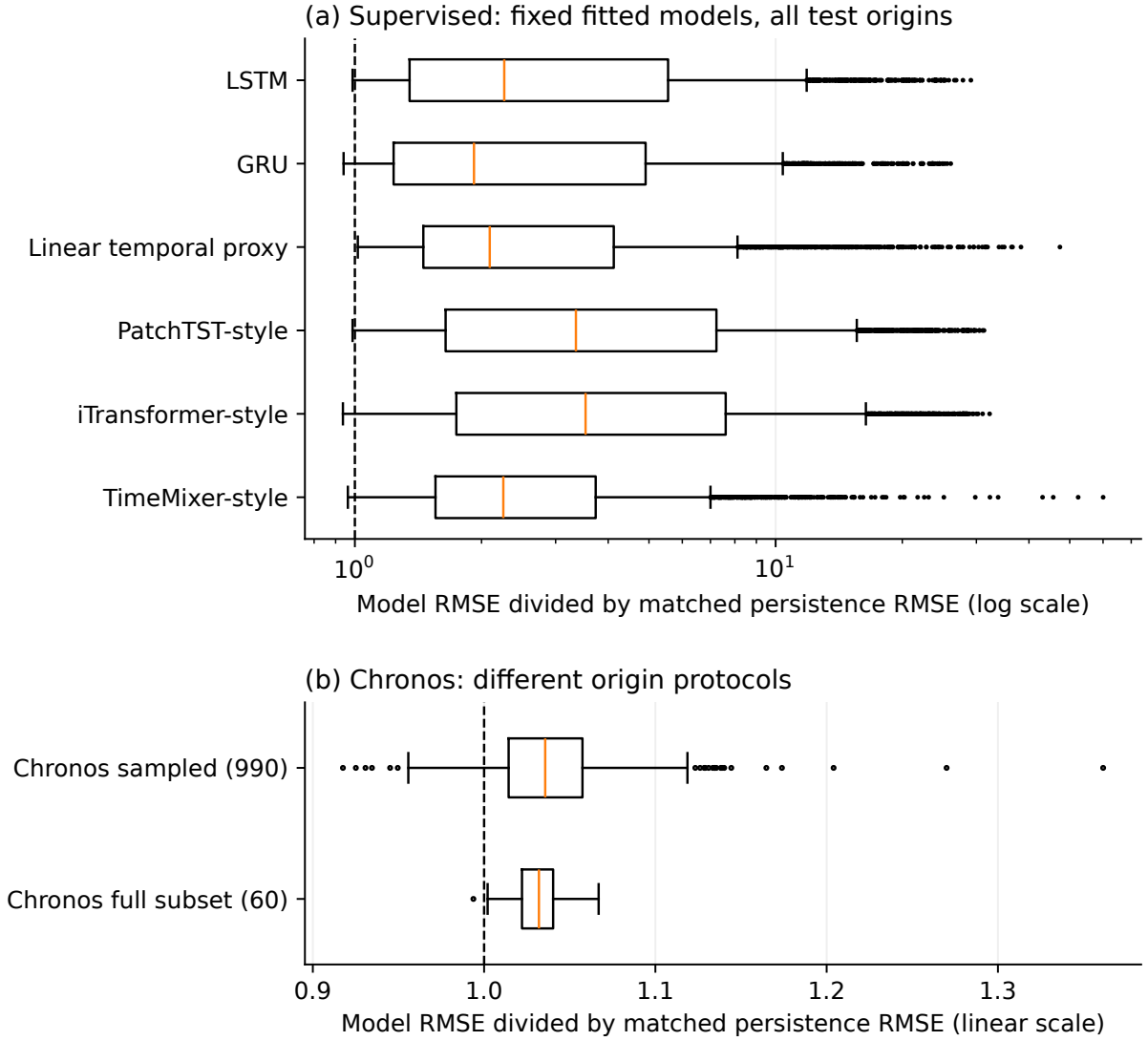


Figure 2: RMSE ratios relative to persistence.

The vertical dashed line marks equality. Panel (a) uses a logarithmic axis and all 35,640 supervised rows. Panel (b) uses a linear axis and separates the 990 sampled-origin Chronos rows from the 60 full-origin subset rows. Boxes show quartiles, the line shows the median, whiskers extend to 1.5 interquartile ranges, and dots show outliers. All ratios remain visible. The protocols differ and rows are dependent.

The full-origin Chronos subset has one RMSE win among 60 rows and one all-three win. Its median ratio is 1.032, also above one. This subset contains the first two asset entries from each list, representing nine unique tickers. It provides a limited sensitivity check rather than a representative test of every asset. Separate generation in the two runs also prevents us from attributing their differences only to origin sampling.

4.3 Patterns across lists

Table 6 separates gate counts by asset list and protocol. Every supervised implementation has a median RMSE ratio above one within each list, although some configurations win. The frequency of local wins varies across lists. These summaries retain list memberships, so a stock that appears in several lists contributes several times. The pattern supports a bounded cross-market result for this selected universe, not a claim that every configuration or every financial forecasting model fails.

Table 6: Gate counts by asset list.

List	Sup. wins	Sup. rows	Chronos wins	Chronos rows	Chronos rate (%)
Dow 30	9	6,480	13	180	7.22
ETF basket	5	2,592	12	72	16.67
Hang Seng large-cap	16	9,288	41	258	15.89
Nasdaq large-tech	0	6,480	14	180	7.78
S&P 100 subset	9	10,800	39	300	13.00

Sup. denotes supervised; the Chronos columns use sampled origins. Each column summarizes its own protocol; differences in rates do not establish a model-family ranking.

5 Discussion

The corrected results support a limited conclusion: most configurations in this experiment do not improve on matched persistence. The interpretation depends on the price-level target, the compact implementations, and the evaluation protocol. The following discussion explains the role of the baseline and the boundaries of the evidence.

5.1 The role of persistence

The persistence forecast starts from the current price level and assumes no change over the forecast horizon. A learned model must estimate a level accurately enough to improve on that simple rule. Lag or smoothing errors may help explain why a visually plausible prediction still has higher RMSE. The present experiment does not isolate those mechanisms, so they remain possible explanations rather than demonstrated causes.

The rare wins also need a precise interpretation. The 39 supervised RMSE wins are configuration–seed rows, not 39 independent assets. Twenty rows improve all three error measures, which is a stricter descriptive criterion. Neither count establishes a generally superior forecasting system, because related rows reuse assets, targets, and observations.

5.2 The scope of the negative result

Every supervised implementation has a median RMSE ratio above one, and Chronos-T5-small also has a median above one within its sampled-origin protocol. These summaries describe the fixed

experiment grid. They do not identify the performance of official full-capacity architectures, larger foundation models, or alternative training searches. The different origins, context lengths, and input variables also prevent a direct model-family ranking.

The target further limits the conclusion. The study evaluates future absolute closing prices rather than returns, direction, volatility, portfolio rankings, or trading profit. Those tasks need their own loss functions, benchmarks, and evaluation designs. The result therefore supports caution about forecasting claims for this price-level task, rather than a claim that financial forecasting is impossible.

5.3 Implications for evaluation

For price-level studies, we recommend reporting persistence alongside learned-model comparisons. Each comparison should state its forecast origins, target dates, training information, and checkpoint-selection rule. A claimed gain from indicators or architecture also needs a design that holds the relevant evaluation conditions constant. When gains occur only in particular assets, horizons, or seeds, the conclusion should retain those conditions.

The correction also shows why dated forecasts matter. An error ratio can be arithmetically correct while comparing model and baseline errors from different test samples. The corrected workflow stores both forecasts on the same dates and verifies their metrics against the original price records. This improves traceability, but it does not remove the remaining data-availability, selection, and statistical-dependence limits.

6 Limitations

The model scope is limited to the selected compact implementations and one small Chronos variant. The linear temporal proxy lacks the decomposition in published DLinear, and the modern style implementations are not official reproductions. Larger models and broader tuning may produce different results. The current experiment therefore cannot establish a ranking of official architectures or a general limit on neural forecasting.

The protocols also limit comparisons between model groups and feature sets. Each feature set retains its own usable-data split, while Chronos uses a longer univariate context and sampled test origins. The full-origin Chronos check contains only nine unique tickers selected by list order. Its results do not prove that sampled-origin estimates are unbiased across the full universe. Repeated assets and related configurations also prevent us from treating the result rows as independent evidence about a wider population.

The data represent a retrospective snapshot rather than a point-in-time market database. Adjusted-price inputs use the downloaded adjustment factors, whose availability at historical forecast origins remains unverified. The configured asset lists do not reconstruct historical constituents or include every delisted asset, so selection and survivorship effects may matter. Overlapping lists also give some tickers more weight in the aggregate summaries. These limitations restrict claims about historical market-wide performance and real-time deployment.

The corrected batch has stronger provenance than the original archive, but it is not an exact recreation of the original weights. The original archive lacks the complete environment, generation state, prediction traces, and checkpoints needed for that claim. Several corrections and runtime differences accompany the rerun, so we cannot attribute every old–new change to baseline alignment alone. Zero-shot use also does not by itself prove that the pretrained model's training data are wholly separate from this study's assets. Finally, our conclusions concern price-level errors and do not establish results for returns, direction, risk, portfolios, or trading profitability.

7 Reproducibility materials

The corrected release stores the source snapshot, arguments, input hashes, package versions, and model-level result files. It includes dated predictions, persistence forecasts, and the table and figure sources. The original local batch retains the fitted checkpoints and feature scalers; the public reproduction scripts regenerate these objects. Complete experiment keys identify the dataset, ticker, price mode, horizon, feature set, implementation, window, and seed where applicable. The boundary export records actual dates and sample counts for each condition rather than using one ticker to represent an entire dataset. These files support inspection of both the reported results and their evaluation scope.

The earlier technical-feature branch references persistence errors from a different test period. Direct tracing of the unchanged archived functions identifies this mismatch in 990 conditions, affecting 17,820 supervised rows. The corrected implementation recomputes persistence on each model's test origins and removes training and validation labels that cross the next segment boundary. We retain the original archive separately and use the complete corrected batch for the revised tables. Because the rerun includes other corrections and runtime differences, we do not attribute all numerical changes to the baseline repair.

The completed batch contains 35,640 supervised rows, 990 sampled-origin Chronos rows, and 60 full-origin subset rows. Verification reconstructs the declared test origins, matches their targets to the input data, and recomputes the three errors, ratios, and gates. Additional checks independently reconstruct all 1,980 supervised split conditions and reconcile the aggregate tables. Inference replay covers 72 stratified checkpoints; this is not a replay or independent retraining of every model. The supplementary material documents these checks and their limits.

The manifests record Python 3.13.3 and PyTorch 2.11.0 with CUDA 12.8 for this batch. Chronos uses the fixed model revision recorded in those manifests rather than an unversioned download. The files retain the original source key `dlinear` for traceability, while the revised manuscript calls that implementation a linear temporal proxy. These records identify the corrected batch without claiming identical behavior in every software or hardware environment.

8 Data and code availability

Yahoo Finance provides the daily prices used in this study. Release v0.2.1 contains the corrected code, exact input snapshot, complete result tables, and dated forecast files at `https://doi.org/10.5281/zenodo.24168592`. The matching GitHub release is available at `https://github.com/Jack-TCN/stock-price-persistence-audit/releases/tag/v0.2.1`. The concept DOI `https://doi.org/10.5281/zenodo.20412777` identifies all archived versions. The earlier v0.1.2 archive at `https://doi.org/10.5281/zenodo.20412970` preserves the pre-correction evaluation. The public release excludes fitted checkpoints and scaler objects; the original local batch retains them, and full reproduction regenerates them.

9 Conclusion

This study compares selected compact supervised forecasters and Chronos-T5-small with latest-close persistence for cross-market stock-price level prediction. Only 39 of 35,640 supervised configuration–seed rows achieve lower RMSE than matched persistence, and every supervised implementation has a median RMSE ratio above one. Chronos-T5-small passes in 119 of 990 sampled-origin rows, but its median ratio of 1.036 also remains above one. A small full-origin subset gives one pass in 60 rows and provides only a limited sensitivity check. For this target and these protocols, the results support baseline-first evaluation: claims of improvement should first establish lower error than persistence on the same forecast origins and target dates.

References

[1] W. Bao, J. Yue, and Y. Rao, "A deep learning framework for financial time series using stacked autoencoders and long-short term memory," *PLOS ONE*, vol. 12, no. 7, article e0180944, 2017. DOI: 10.1371/journal.pone.0180944.

[2] J. Patel, S. Shah, P. Thakkar, and K. Kotecha, "Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques," *Expert Systems with Applications*, vol. 42, no. 1, pp. 259–268, 2015. DOI: 10.1016/j.eswa.2014.07.040.

[3] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, "Financial time series forecasting with deep learning: A systematic literature review: 2005–2019," *Applied Soft Computing*, vol. 90, article 106181, 2020. DOI: 10.1016/j.asoc.2020.106181.

[4] A. M. Ozbayoglu, M. U. Gudelek, and O. B. Sezer, "Deep learning for financial applications: A survey," *Applied Soft Computing*, vol. 93, article 106384, 2020. DOI: 10.1016/j.asoc.2020.106384.

[5] J. Y. Campbell and S. B. Thompson, "Predicting excess stock returns out of sample: Can anything beat the historical average?" *Review of Financial Studies*, vol. 21, no. 4, pp. 1509–1531, 2008. DOI: 10.1093/rfs/hhm055.

[6] I. Welch and A. Goyal, "A comprehensive look at the empirical performance of equity premium prediction," *Review of Financial Studies*, vol. 21, no. 4, pp. 1455–1508, 2008. DOI: 10.1093/rfs/hhm014.

[7] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. OTexts, 2018. `https://otexts.com/fpp2/`.

[8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," *PLOS ONE*, vol. 13, no. 3, article e0194889, 2018. DOI: 10.1371/journal.pone.0194889.

[9] B. Lim and S. Zohren, "Time-series forecasting with deep learning: A survey," *Philosophical Transactions of the Royal Society A*, vol. 379, no. 2194, article 20200209, 2021. DOI: 10.1098/rsta.2020.0209.

[10] K. Benidis et al., "Deep learning for time series forecasting: Tutorial and literature survey," *ACM Computing Surveys*, vol. 55, no. 6, 2023 (first published online in 2022). DOI: 10.1145/3533382.

[11] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, pp. 11121–11128, 2023. DOI: 10.1609/aaai.v37i9.26317.

[12] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *International Conference on Learning Representations*, 2023. Author preprint.

[13] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "iTransformer: Inverted transformers are effective for time series forecasting," in *International Conference on Learning Representations*, 2024. Author preprint.

[14] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J. Y. Zhang, and J. Zhou, "TimeMixer: Decomposable multiscale mixing for time series forecasting," in *International Conference on Learning Representations*, 2024. Author preprint.

[15] A. F. Ansari et al., "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024. Publisher page.

[16] K. Rasul et al., "Lag-Llama: Towards foundation models for probabilistic time series forecasting," *arXiv preprint arXiv:2310.08278v3*, 2024. `https://arxiv.org/abs/2310.08278v3`.

[17] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," in *Proceedings of the 41st International Conference on Machine Learning*, PMLR, vol. 235, pp. 10148–10167, 2024. Publisher page.

[18] M. Goswami, K. Szafer, A. Choudhry, Y. Cai, S. Li, and A. Dubrawski, "MOMENT: A family of open time-series foundation models," in *Proceedings of the 41st International Conference on Machine Learning*, PMLR, vol. 235, pp. 16115–16152, 2024. Publisher page.

[19] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified training of universal time series forecasting transformers," in *Proceedings of the 41st International Conference on Machine Learning*, PMLR, vol. 235, pp. 53140–53164, 2024. Publisher page.